



Reverse Level Order Traversal (easy)

We'll cover the following ^

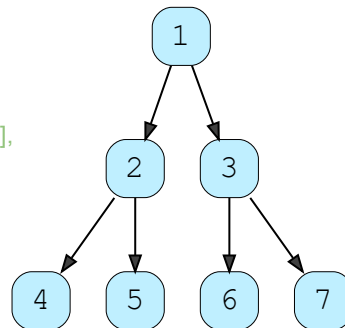
- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

Given a binary tree, populate an array to represent its level-by-level traversal in reverse order, i.e., the **lowest level comes first**. You should populate the values of all nodes in each level from left to right in separate sub-arrays.

Example 1:

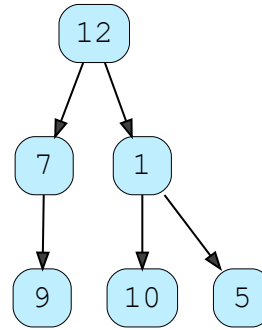
Reverse Level Order Traversal: `[[4,5,6,7], [2,3], [1]]`



Example 2:



Reverse Level Order Traversal: [[9,10,5],
[7,1],
[12]]



Try it yourself

Try solving this question here:



Java



Python3



JS



C++

```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 };
12
13 class ReverseLevelOrderTraversal {
14     public static List<List<Integer>> traverse(TreeNode root) {
15         List<List<Integer>> result = new LinkedList<List<Integer>>();
16         // TODO: Write your code here
17         return result;
18     }
19
20     public static void main(String[] args) {
21         TreeNode root = new TreeNode(12);
22         root.left = new TreeNode(7);
23         root.right = new TreeNode(1);
24         root.left.left = new TreeNode(9);
25         root.right.left = new TreeNode(10);
26         root.right.right = new TreeNode(5);
27         List<List<Integer>> result = ReverseLevelOrderTraversal.traverse(root);
28         System.out.println("Reverse level order traversal: " + result);
29     }
30 }
31

```

Run

Save

Reset



Solution

This problem follows the [Binary Tree Level Order Traversal](#) pattern. We can follow the same **BFS** approach. The only difference will be that instead of appending the current level at the end, we will append the current level at the

beginning of the result list.

Code

> Here is what our algorithm will look like; only the highlighted lines have changed. Please note that, for **Java**, we will use a **LinkedList** instead of an **ArrayList** for our result list. As in the case of **ArrayList**, appending an element at the beginning means shifting all the existing elements. Since we need to append the level array at the beginning of the result list, a **LinkedList** will be better, as this shifting of elements is not required in a **LinkedList**. Similarly, we will use a double-ended queue (deque) for **Python**, **C++**, and **JavaScript**.



```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 };
12
13 class ReverseLevelOrderTraversal {
14     public static List<List<Integer>> traverse(TreeNode root) {
15         List<List<Integer>> result = new LinkedList<List<Integer>>();
16         if (root == null)
17             return result;
18
19         Queue<TreeNode> queue = new LinkedList<>();
20         queue.offer(root);
21         while (!queue.isEmpty()) {
22             int levelSize = queue.size();
23             List<Integer> currentLevel = new ArrayList<>(levelSize);
24             for (int i = 0; i < levelSize; i++) {
25                 TreeNode currentNode = queue.poll();
26                 // add the node to the current level
27                 currentLevel.add(currentNode.val);
28                 // insert the children of current node to the queue
29                 if (currentNode.left != null)
30                     queue.offer(currentNode.left);
31                 if (currentNode.right != null)
32                     queue.offer(currentNode.right);
33             }
34             // append the current level at the beginning
35             result.add(0, currentLevel);
36         }
37
38         return result;
39     }
40
41     public static void main(String[] args) {
42         TreeNode root = new TreeNode(12);
43         root.left = new TreeNode(7);
44         root.right = new TreeNode(1);
45         root.left.left = new TreeNode(9);
46         root.right.left = new TreeNode(10);
47         root.right.right = new TreeNode(5);
48         List<List<Integer>> result = ReverseLevelOrderTraversal.traverse(root);
49         System.out.println("Reverse level order traversal: " + result);
50     }
51 }

```

52

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space complexity

The space complexity of the above algorithm will be $O(N)$ as we need to return a list containing the level order traversal. We will also need $O(N)$ space for the queue. Since we can have a maximum of $N/2$ nodes at any level (this could happen only at the lowest level), therefore we will need $O(N)$ space to store them in the queue.

